

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Introduction
- ▶ Classes and Objects
- ▶ Encapsulation
- ▶ Inheritance
- ▶ Polymorphism
- ▶ Abstraction
- ▶ Advanced Inheritance and Polymorphism
- ▶ SOLID principles

Let's Start

Introduction

- ▶ Object-oriented programming (OOP) is a way of developing software applications using real-world terminologies to create entities that interact with one another using objects.
- ▶ OOP makes applications flexible, reusable, well-structured, and easy to debug and test.
- ▶ The basic building blocks of OOP are classes, methods, properties, and objects.
- ▶ Classes define the structure of an object, methods represent the behavior of an object, properties hold the data of an object, and objects are instances of a class.
- ▶ **Here are some of the benefits of OOP:**
- ▶ **Reusability:** Object-oriented programming (OOP) introduces *Classes* and *Objects* to promote code reuse. Instead of repeating the same code, you can create a class once and use its object multiple times wherever needed.

Introduction

- ▶ **Extensibility:** OOP allows you to add new features without rewriting everything. Using concepts like *Inheritance* you can extend existing code easily.
- ▶ **Simplicity:** Unlike traditional modular programming, where code can be scattered and harder to secure, OOP organizes code using *Abstraction*, *Encapsulation*, and *Polymorphism*. This simplifies understanding and enhances security.
- ▶ **Maintainability:** Since OOP focuses on Reusability, Extensibility, and Simplicity, it results in clean, organized, and easy-to-maintain code. This improves the overall quality and lifespan of applications.

Classes and Objects

- ▶ **Class :** A class is a user-defined blueprint that defines both *state* (properties) and *behavior* (actions). It serves as a template for creating objects. In C#, a class consists of a name, attributes (properties), and operations (methods).
- ▶ **Object:** An object is a specific instance of a class. It brings the class to life and performs actions defined by the class. To access a class's members (attributes and methods), you use an object along with the dot(.) Operator.
- ▶ **Methods:** A method represents a particular behavior. It performs some action and might return information about an object, or update an object's data.
- ▶ **Properties:** Properties hold the data temporarily during the execution of an application.

Classes and Objects

► Ex:

```
public class Calculator
{
    public int CalculateSum(int no1, int no2)
    {
        return no1 + no2;
    }
}
```

```
//Creating object
Calculator calObject = new Calculator();

//Accessing Calculator class member using
Calculator class object

int result = calObject.CalculateSum(10, 20);
```

Constructors and Destructors

- ▶ **Constructor:** It is a special method present inside a class responsible for initializing the variables of that class.
- ▶ **Destructor:** A *destructor* is a special method that is called automatically when an object is destroyed or goes out of scope. It is used to release resources. In C#, a destructor has the same name as the class but is preceded by a tilde (~).

Constructors and Destructors

```
using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ExplicitConstructor obj = new ExplicitConstructor();

            Console.ReadKey();
        }
    }
    class ExplicitConstructor
    {
        public ExplicitConstructor()
        {
            Console.WriteLine("Explicit Constructor is Called!");
        }
    }
}
```

Access Modifiers

Modifier	Accessibility
Public	Accessible from anywhere in the application.
Private	Accessible only within the class where it is defined.
Protected	Accessible within the class and by derived classes.
Internal	Accessible within the same assembly (project).
Protected Internal	Accessible within the same assembly or by derived classes.
Private Protected	Accessible within the same class and derived classes, but only in the same assembly.

Encapsulation

- ▶ **Property:** Properties hold the data temporarily during the execution of an application.
- ▶ **Fields :** Fields are variables that directly hold data in a class. Typically kept private to restrict direct access and ensure better control.
- ▶ **Accessors :** An accessor is a special block of code in a property that is used to get (retrieve) or set (assign) the value of a private field in a controlled way.

Get Accessor : The get accessor is used to retrieve the value of a private field.

Set Accessor : The set accessor is used to assign a value to a private field.

Encapsulation

```
class Product
{
    // Private field
    private string name;

    // Property with get and set accessors
    public string Name
    {
        get { return name; } // Get accessor
        set
        {
            if (!string.IsNullOrEmpty(value)) // Validation
                name = value;
            else
                Console.WriteLine("Name cannot be empty.");
        }
    }

    // Read-only property
    public string Description
    {
        get { return $"Product Name: {Name}"; }
    }

    // Constructor to initialize the field
    public Product(string productName)
    {
        Name = productName; // Uses the property
    }
}
```

```
class Program
{
    static void Main()
    {
        // Create an object
        Product product = new Product("Laptop");

        // Access and modify properties
        Console.WriteLine(product.Name); // Get accessor
        product.Name = "Smartphone";    // Set accessor
        Console.WriteLine(product.Description); // Read-only property
        //product.Description = "It is fastest smartphone"; // Error - read-only property
        // Try invalid value
        product.Name = ""; // Validation fails
    }
}
```

Encapsulation

- ▶ Encapsulation Hides the internal state and functionality of an object and only allows access through a public set of functions.
- ▶ The Encapsulation Principle in C# is very similar to a Capsule. As a capsule binds its medicine within it, in the same way in C#, the Encapsulation Principle binds the State (Variables) and Behaviour (Methods) into a single unit called class, enum, interface, etc.
- ▶ Encapsulation hides the data and implementation details show only the required members within a class, thus hiding complexity from other code. No other code needs to know about implementation detail and also can't modify the code of the class's data and methods
- ▶ Most object-oriented programming languages allow you to create classes and their properties and methods along with the access modifiers such as public, private, protected, and internal to show or hide data members and implementation details. Interfaces and abstract classes can also be used for encapsulation.

Encapsulation

► Advantages of Encapsulation:

- Hides data and complexities.
- Restrict unauthorized access of data by allowing authorization before data access.
- Allow validation before setting data.
- Only the author of the class needs to understand the implementation, not others.
- Makes applications easy to maintain.